```
from datetime import datetime
import random, time, math
from copy import deepcopy, copy
import decimal

class Board:
    def __init__(self, queen_count=8):
        self.queen_count = queen_count
        self.reset()

    def reset(self):
        self.queens = [-1 for i in range(0, self.queen_count)]

        for i in range(0, self.queen_count):
            self.queens[i] = random.randint(0, self.queen_count - 1)
            # self.queens[row] = column


    def calculateCost(self):
        threat = 0

        for queen in range(0, self.queen_count):
            for next_queen in range(queen+1, self.queen_count):
                if self.queens[queen] == self.queens[next_queen] or abs(queen - next_queen) == abs(self.queens[queen] - self.que
                    threat += 1

        return threat

    @staticmethod
    def calculateCostWithQueens(queens):
        threat = 0
        queen_count = len(queens)

        for queen in range(0, queen_count):
            for next_queen in range(queen+1, queen_count):
                if queens[queen] == queens[next_queen] or abs(queen - next_queen) == abs(queens[queen] - queens[next_queen]):
                    threat += 1

        return threat

    @staticmethod
    def toString(queens):
        board_string = ""

        for row, col in enumerate(queens):
            board_string += "(%s, %s)\n" % (row, col)

        return board_string

    def getLowerCostBoard(self):
        displacement_count = 0
        temp_queens = self.queens
        lowest_cost = self.calculateCost(temp_queens)

        for i in range(0, self.queen_count):
            temp_queens[i] = (temp_queens[i] + 1) % (self.queen_count - 1)

            for j in range(queen+1, self.queen_count):
                temp_queens[j] = (temp_queens[j] + 1) % (self.queen_count - 1)

    def __str__(self):
        board_string = ""

        for row, col in enumerate(self.queens):
            board_string += "(%s, %s)\n" % (row, col)

        return board_string

class SimulatedAnnealing:
    def __init__(self, board):
        self.elapsedTime = 0;
        self.board = board
        self.temperature = 4000
        self.sch = 0.99
        self.startTime = datetime.now()
```

```
    def run(self):
        board = self.board
        board_queens = self.board.queens[:]
        solutionFound = False

        for k in range(0, 170000):
            self.temperature *= self.sch
            board.reset()
            successor_queens = board.queens[:]
            dw = Board.calculateCostWithQueens(successor_queens) - Board.calculateCostWithQueens(board_queens)
            exp = decimal.Decimal(decimal.Decimal(math.e) ** (decimal.Decimal(-dw) * decimal.Decimal(self.temperature)))

            if dw > 0 or random.uniform(0, 1) < exp:
                board_queens = successor_queens[:]

            if Board.calculateCostWithQueens(board_queens) == 0:
                print("Solution:")
                print(Board.toString(board_queens))
                self.elapsedTime = self.getElapsedTime()
                print("Success, Elapsed Time: %sms" % (str(self.elapsedTime)))
                solutionFound = True
                break

        if solutionFound == False:
            self.elapsedTime = self.getElapsedTime()
            print("Unsuccessful, Elapsed Time: %sms" % (str(self.elapsedTime)))

        return self.elapsedTime

    def getElapsedTime(self):
        endTime = datetime.now()
        elapsedTime = (endTime - self.startTime).microseconds / 1000
        return elapsedTime


if __name__ == '__main__':
    board = Board()
    print("Board:")
    print(board)
    SimulatedAnnealing(board).run()
```

```
Board:
(0, 0)
(1, 2)
(2, 3)
(3, 7)
(4, 3)
(5, 2)
(6, 3)
(7, 2)

Unsuccessful, Elapsed Time: 238.565ms
```